Experiment 7

API Integration with Zoho Creator

Aim

To integrate Zoho Creator with external applications and services using Application Programming Interfaces (APIs) to enable secure data exchange, automate business processes and extend the functionality of the Vehicle Service & Repair Request Management system.

Theory

API integration allows applications to communicate and exchange data with external systems, enabling developers to retrieve, create, update and manage records between applications. In the Vehicle Service application, APIs are used to read service request data into Zoho Analytics for dashboards and to call external services from Deluge using connections and **invokeUrl**.

Procedure

1. Log in to Zoho Creator and open the Vehicle Service & Repair Request Management application.
2. Identify the external application or service to integrate (Zoho Analytics for reporting, and the Creator REST API for data exchange).
3. Configure the required API access and authentication settings (OAuth connection in Zoho Creator).
4. Generate or configure the necessary API credentials and scopes.
5. Use supported integration options or Deluge functions to send and receive data.
6. Configure the request URL, HTTP method, headers and required parameters.
7. Test the integration and verify that data is exchanged correctly.
8. Handle errors and validate responses to ensure reliable operation.

Applications of API Integration in this Project

- Sending the Service_Request_Form data to Zoho Analytics for charts and dashboards.
- Sending automated email or SMS notifications when status changes.
- Synchronizing service requests with an external spare-parts inventory system.
- Integrating payment gateways for the Net Service Cost Payable.
- Retrieving external data (for example vehicle model details) for reports.

Request Details

Item	Value
Method	GET
Target	Service_Request_Report through the Creator REST API
Authentication	OAuth connection created in Zoho Creator (read scope on the report)
Used for	Reading service requests for reports, dashboards and external systems

Sample Deluge: Fetching Records Through a Connection

Replace the placeholders with your own account owner name and connection link name.

```
baseUrl = "https://creator.zoho.in/api/v2/<account_owner>/";
response = invokeUrl
[
    url :baseUrl + "vehicle-service-repair-request-management/report/Service_Request_Report"
    type :GET
    connection :"creator_connection"
];
info response;
if(response.get("code") == 3000)
{
    records = response.get("data");
    info "Records fetched: " + records.size();
}
else
{
    info "API error: " + response;
}
```

Process Diagram

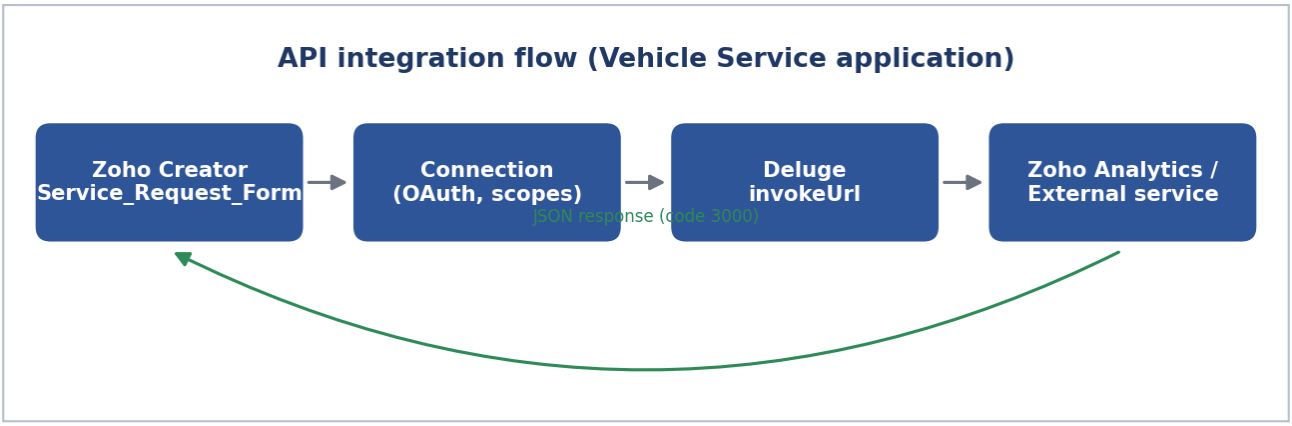


Figure 1: Diagram of the API integration flow (illustrative diagram, not a screenshot).

Expected Response Format (Illustrative)

```
{
  "code": 3000,
  "data": [
    {
      "Owner_Name": { "first_name": "Ramesh", "last_name": "Kannan" },
      "Vehicle_Registration_Number": "TN 34 AB 5678",
      "Vehicle_Type": "Hatchback",
      "Service_Type": "Breakdown Repair"
    }
  ]
}
```

Error Handling Checklist

- Check that the response code is 3000 before using the data.
- Log the full response when the code is different and notify the administrator.
- Keep credentials in the connection; never write tokens inside the script.
- Fetch only the required records using criteria to avoid slow calls.

Metrics Built in Zoho Analytics from the Sample Data

Sample data for this lab record. Values shown are the expected output for these sample records.

Metric	Value
Total service requests	5
Average estimated cost	₹4,840
Requests by service type	Routine 2, Breakdown 2, Accident 1
Requests by vehicle type	Hatchback 2, Sedan 2, SUV 1

Result

Zoho Creator was integrated with Zoho Analytics and the Creator REST API for the Vehicle Service application, and data exchange was verified.

Conclusion

API integration in Zoho Creator enables communication between applications and external services. It supports data exchange, workflow automation and improved operational efficiency. By configuring APIs with appropriate authentication, validation and error handling, developers can build more flexible and connected applications.